

# DSAA3073: Theories in Data Science

## Week 9: Online Learning II: Online Convex Optimization & FTRL

---

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-06-27

Explorative projection

# Online Convex Optimization

We've conquered the expert advice problem with both discrete and continuous losses. But all our algorithms assume we're choosing from a **finite** set of  $n$  experts. What if we face an **infinite** decision space?

**The continuous challenge:** MWU handles discrete experts. What about continuous decision spaces? This leads to **online convex optimization** (OCO, a framework where decisions come from an infinite continuous set rather than a finite list of experts). Many real problems have continuous parameters: portfolio weights (any vector on the probability simplex), neural network parameters (high-dimensional vectors), control inputs (continuous settings).

**The remarkable universality:** Transitioning from discrete to continuous spaces, the same multiplicative principle applies. Instead of exponentially reweighting  $n$  experts, we'll use gradient-based updates in continuous space. And astonishingly, the  $O(\sqrt{T})$  regret rate persists—revealing a deep universality: whether discrete or continuous, adversarial online learning requires  $\sqrt{T}$  cumulative regret.

**Progress on backbone:** OCO connects online learning to continuous optimization, bridging the gap between the discrete expert-tracking algorithms we've studied and the gradient-based methods that power modern machine learning. This framework will directly inform neural network training in Week 11.

## ← From Week 4: Convexity and Optimization

Week 4 introduced convex functions and gradient descent for batch optimization. Online convex optimization extends this to sequential adversarial losses. The key difference: batch GD minimizes a fixed function; online GD adapts to a sequence of functions revealed one at a time.

## ← From Week 7: Boosting II: AdaBoost and Loss Minimization

Week 7 showed AdaBoost performs coordinate descent on exponential loss. This week we'll see AdaBoost is actually a special case of the more general Follow-The-Regularized-Leader (FTRL) framework with negative entropy regularization.

## Framework

**Motivation:** Many problems have continuous decision spaces: portfolio allocation (simplex in  $\mathbb{R}^n$ ), parameter tuning (boxes in  $\mathbb{R}^d$ ), control (Euclidean balls). We need online learning for infinite action spaces, not just finite expert sets.

**Concrete example first:** Consider online linear regression. Each round, you observe a feature vector (e.g., house size, location score) and must predict the price using weights  $x \in \mathbb{R}^d$ . After your prediction, the true price is revealed and you incur squared error. Your decision space is all possible weight vectors in some bounded region—infinitely many choices, not just  $n$  discrete experts.

**Definition: Online Convex Optimization**

- Decision space: closed convex set  $\mathcal{K} \subseteq \mathbb{R}^d$  (where  $\mathcal{K}$  is a convex set; a line segment between two points in  $\mathcal{K}$  is also in  $\mathcal{K}$ —examples: balls, boxes, simplices, halfspaces)
- Protocol for  $t = 1, \dots, T$ :
  1. Learner chooses  $x_t \in \mathcal{K}$
  2. Environment reveals convex loss function  $f_t : \mathcal{K} \rightarrow \mathbb{R}$  (a function that maps each decision to a loss value)
  3. Learner incurs loss  $f_t(x_t)$
- Regret:

$$R_T = \sum_{t=1}^T f_t(x_t) - \min_{x \in \mathcal{K}} \sum_{t=1}^T f_t(x) \quad (1)$$

Online convex optimization generalizes expert advice: each point  $x \in \mathcal{K}$  is like an “expert”, but now we have infinitely many of them!

**Example: Portfolio Optimization**

- $\mathcal{K} = \{x \in \mathbb{R}^n : x_i \geq 0, \sum_i x_i = 1\}$  (simplex of portfolio allocations)
- $x_t$  = portfolio at day  $t$
- $f_t(x) = -\langle r_t, x \rangle$  where  $r_t$  = return vector on day  $t$
- Goal: compete with best fixed portfolio in hindsight

 **Productive Struggle**

**Challenge:** Before reading Online Gradient Descent, design your own algorithm for online convex optimization.

**Setup:** Decision space  $\mathcal{K} = \{x \in \mathbb{R}^d : \|x\| \leq 1\}$  (unit ball), convex losses  $f_t$ .

**Your task (10-15 minutes):**

1. You observe gradient  $\nabla f_t(x_t)$  after playing  $x_t$ . How should you update?
2. What if the update takes you outside  $\mathcal{K}$ ?
3. What step size should you use?
4. How does this relate to regular gradient descent?

**Guiding questions:**

- In batch GD, we do  $x \leftarrow x - \eta \nabla f(x)$ . Can we use the same rule here?
- What happens if we use a constant step size  $\eta$  for all rounds?
- Should the step size depend on  $T$ ? On properties of  $\mathcal{K}$  and  $f_t$ ?

**After struggling:** Compare your approach to OGD below. Did you discover projection? Did you find the right step size scaling?

**Online Gradient Descent (OGD)**

The natural extension of gradient descent to the online setting.

### Definition: Online Gradient Descent

**Parameters:** Step size  $\eta_t > 0$

**Initialization:** Choose  $x_1 \in \mathcal{K}$  arbitrarily

**For each round  $t = 1, \dots, T$ :**

1. **Play:**  $x_t$
2. **Observe:**  $f_t$  and compute gradient  $\nabla f_t(x_t)$
3. **Update:**

$$\tilde{x}_{t+1} = x_t - \eta_t \nabla f_t(x_t) \quad (2)$$

$$x_{t+1} = \Pi_{\mathcal{K}}(\tilde{x}_{t+1}) \quad (3)$$

where  $\Pi_{\mathcal{K}}$  is the **Euclidean projection** onto  $\mathcal{K}$  (the closest point in  $\mathcal{K}$  to the given point, measured by Euclidean distance:  $\Pi_{\mathcal{K}}(y) = \arg \min_{x \in \mathcal{K}} \|x - y\|$ ).

The projection step  $\Pi_{\mathcal{K}}$  ensures we stay in the feasible set. For many common sets (simplex, ball, box), projection has closed-form solutions.

**Lemma:** for any point  $y$  and any  $x^* \in \mathcal{K}$ ,

$$\|\Pi_{\mathcal{K}}(y) - x^*\| \leq \|y - x^*\| \quad (4)$$

This says projecting onto  $\mathcal{K}$  never increases distance to points in  $\mathcal{K}$ . This property is crucial for proving the OGD regret bound.

### Intuition:

- Move in direction of negative gradient (steepest descent)
- Project back onto constraint set if necessary
- Step size  $\eta_t$  controls how far we move

## Regret Analysis

**Motivation:** The OGD proof uses a different technique than MWU's potential function. Instead, we use a **telescoping argument**: relate each round's regret to the distance  $\|x_t - x^*\|^2$ , which telescopes over rounds. This geometric approach is more natural for continuous spaces.

### Proof Template: Telescoping Distance Arguments

This proof pattern appears throughout online convex optimization:

1. **Use convexity:**  $f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle$
2. **Relate gradient to update:**  $\nabla f_t(x_t) = \frac{x_t - \tilde{x}_{t+1}}{\eta}$
3. **Apply identity:**  $2\langle a, b \rangle = \|a\|^2 + \|b\|^2 - \|a - b\|^2$
4. **Use projection property:**  $\|x_{t+1} - x^*\|^2 \leq \|\tilde{x}_{t+1} - x^*\|^2$
5. **Telescope:** Sum over  $t$  to get cancellation
6. **Optimize step size:** Balance two terms

**When to use:** Gradient-based online algorithms, projections, convex losses

**Theorem: Online Gradient Descent Regret Bound**

The  $O(\sqrt{T})$  regret matches MWU!  
This is the optimal rate for online  
convex optimization.

Assume:

- **Convex losses:**  $f_t : \mathcal{K} \rightarrow \mathbb{R}$  is convex for all  $t = 1, \dots, T$
- **Bounded gradients:**  $\|\nabla f_t(x)\| \leq G$  for all  $t \in [T]$  and all  $x \in \mathcal{K}$
- **Bounded diameter:**  $\text{diam}(\mathcal{K}) = \max_{x, y \in \mathcal{K}} \|x - y\| \leq D$
- **Constant step size:**  $\eta_t = \eta > 0$  for all  $t$

Then for any comparator  $x^* \in \mathcal{K}$ :

$$R_T = \sum_{t=1}^T f_t(x_t) - \sum_{t=1}^T f_t(x^*) \leq \frac{D^2}{2\eta} + \frac{\eta G^2 T}{2} \quad (5)$$

**Optimal tuning:** Setting  $\eta = \frac{D}{G\sqrt{T}}$  gives:

$$R_T \leq DG\sqrt{T} \quad (6)$$

This achieves the optimal  $O(\sqrt{T})$  regret rate for online convex optimization.

**Proof. Step 1: Decompose Regret**

For any  $x^* \in \mathcal{K}$ :

$$f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle \quad (7)$$

by convexity of  $f_t$ .

Summing over  $t$ :

$$R_T \leq \sum_{t=1}^T \langle \nabla f_t(x_t), x_t - x^* \rangle \quad (8)$$

**Step 2: Relate to Weight Updates**

From the update rule  $\tilde{x}_{t+1} = x_t - \eta \nabla f_t(x_t)$ :

$$\nabla f_t(x_t) = \frac{x_t - \tilde{x}_{t+1}}{\eta} \quad (9)$$

Therefore:

$$\langle \nabla f_t(x_t), x_t - x^* \rangle = \frac{1}{\eta} \langle x_t - \tilde{x}_{t+1}, x_t - x^* \rangle \quad (10)$$

**Step 3: Telescoping Argument**

Using the identity  $2\langle a, b \rangle = \|a\|^2 + \|b\|^2 - \|a - b\|^2$  (a polarization identity that relates inner products to norms—it can be verified by expanding the squared norms):

$$2\langle x_t - \tilde{x}_{t+1}, x_t - x^* \rangle = \|x_t - x^*\|^2 + \|x_t - \tilde{x}_{t+1}\|^2 - \|\tilde{x}_{t+1} - x^*\|^2 \quad (11)$$

Since  $x_{t+1} = \Pi_{\mathcal{K}}(\tilde{x}_{t+1})$  and projection is non-expansive:

$$\|x_{t+1} - x^*\|^2 \leq \|\tilde{x}_{t+1} - x^*\|^2 \quad (12)$$

Euclidean projection onto a closed convex set is the closest feasible point, so it cannot be farther from any feasible comparator  $x^*$  than the unprojected point.

Therefore:

$$\langle \nabla f_t(x_t), x_t - x^* \rangle \leq \frac{\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2}{2\eta} + \frac{\eta \|\nabla f_t(x_t)\|^2}{2} \quad (13)$$

#### Step 4: Sum Over Rounds

$$\begin{aligned} R_T &\leq \sum_{t=1}^T \left[ \frac{\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2}{2\eta} + \frac{\eta G^2}{2} \right] \\ &= \frac{\|x_1 - x^*\|^2 - \|x_{T+1} - x^*\|^2}{2\eta} + \frac{\eta G^2 T}{2} \\ &\leq \frac{D^2}{2\eta} + \frac{\eta G^2 T}{2} \end{aligned} \quad (14)$$

The last step uses  $\|x_1 - x^*\|^2 \leq D^2$  and drops the negative term.

#### Step 5: Optimize $\eta$

Minimize  $\frac{D^2}{2\eta} + \frac{\eta G^2 T}{2}$  by taking derivative:

$$-\frac{D^2}{2\eta^2} + \frac{G^2 T}{2} = 0 \implies \eta = \frac{D}{G\sqrt{T}} \quad (15)$$

Substituting back:

$$R_T \leq DG\sqrt{T} \quad (16)$$

■

#### ✓ Checkpoint

**Checkpoint 7:** The OGD regret bound has two terms:  $\frac{D^2}{2\eta}$  and  $\frac{\eta G^2 T}{2}$ . Why does the first term decrease with  $\eta$  while the second increases?

**Answer:**  $\frac{D^2}{2\eta}$  measures “initialization regret”—how far we start from the optimum. Larger  $\eta$  means faster initial movement (smaller regret).  $\frac{\eta G^2 T}{2}$  measures “gradient noise accumulation”—too-large steps overshoot. The optimal  $\eta$  balances these tradeoffs.

**Example: Online Linear Regression**

- $\mathcal{K} = \{x \in \mathbb{R}^d : \|x\| \leq B\}$  (ball of radius  $B$ )
- At time  $t$ : observe feature vector  $a_t \in \mathbb{R}^d$ , predict  $\langle x_t, a_t \rangle$ , observe true label  $b_t$
- Loss:  $f_t(x) = (\langle x, a_t \rangle - b_t)^2$
- Gradient:  $\nabla f_t(x_t) = 2(\langle x_t, a_t \rangle - b_t)a_t$

OGD achieves regret  $O(\sqrt{T})$  compared to best fixed predictor, assuming  $\|a_t\|$  and  $|b_t|$  are bounded so the gradients are bounded.

**🧠 Mental Model: Comparison: Discrete vs Continuous Online Learning**

The journey from expert advice to convex optimization:

**Discrete Expert Advice:**

- Decision space:  $\{1, 2, \dots, n\}$  (finite)
- Algorithm: MWU with exponential weights
- Regret:  $O(\sqrt{T \ln n})$
- Update:  $w_i \leftarrow w_i(1 - \eta m_{t(i)})$
- Geometry: Probability simplex with entropy

**Continuous Convex Optimization:**

- Decision space:  $\mathcal{K} \subseteq \mathbb{R}^d$  (infinite)
- Algorithm: OGD with gradient descent
- Regret:  $O(\sqrt{T})$  (no  $\ln n$  factor!)
- Update:  $x \leftarrow \Pi_{\mathcal{K}}(x - \eta \nabla f_t(x))$
- Geometry: Euclidean space with quadratic regularization

**Bridge: FTRL Framework:**

- Unifies both through regularization
- Entropy regularizer  $\rightarrow$  MWU
- Quadratic regularizer  $\rightarrow$  OGD
- Same proof technique: stability via strong convexity

**Why the difference?**

- Discrete:  $\ln n$  factor from covering  $n$  experts
- Continuous: No discrete experts, but need projection
- Both:  $\sqrt{T}$  from fundamental information limit

**← From Week 1: Infinite Hypothesis Classes**

Week 1 asked: Can we learn with infinite hypothesis classes? PAC learning said “yes, if VC dimension is finite.” Online learning gives a different answer: even with infinitely many “experts” (points in  $\mathcal{K}$ ), we achieve  $O(\sqrt{T})$  regret via gradient descent. The finiteness condition shifts from VC dimension to strong convexity + Lipschitz gradients.

**Connection****OGD vs. Batch Gradient Descent:**

- Batch GD: minimize fixed objective  $f(x) = \left(\frac{1}{n}\right) \sum_{i=1}^n f_i(x)$
- Online GD: minimize regret against sequence of functions  $f_1, f_2, \dots, f_T$
- Batch GD converges to optimum; OGD competes with best fixed point
- In stochastic setting (i.i.d. losses), OGD = SGD!

**Projection onto Common Sets**

For OGD to be practical, we need efficient projections:

| Set $\mathcal{K}$                   | Projection $\Pi_{\mathcal{K}}(y)$ |
|-------------------------------------|-----------------------------------|
| Euclidean ball: $\ x\  \leq R$      | $\frac{Ry}{\max(R, \ y\ )}$       |
| Box: $a_i \leq x_i \leq b_i$        | $(\min(\max(y_i, a_i), b_i))_i$   |
| Simplex: $x \geq 0, \sum_i x_i = 1$ | Sorting algorithm, $O(d \log d)$  |
| Positive orthant: $x \geq 0$        | $(\max(y_i, 0))_i$                |

→ **Week 11 Preview: Projection in Neural Network Training**

Week 11's neural networks often use constrained optimization: weight clipping for stability, normalization layers (constraining weights to unit norm), dropout (implicit constraint on capacity). The projection operation  $\Pi_{\mathcal{K}}$  we use in OGD appears throughout deep learning, though often implicitly through regularization or architecture design rather than explicit projection steps.

**Key Takeaway**

Online Gradient Descent extends online learning to continuous domains, achieving  $O(\sqrt{T})$  regret for convex losses. The key ingredients are: gradient-based updates, projection onto the constraint set, and careful step size selection. OGD unifies batch optimization and online learning.

→ **Week 11 Preview: Stochastic Gradient Descent**

Week 11's neural network training uses SGD, which is OGD in the stochastic setting! When losses  $f_t$  are i.i.d. samples from a distribution, OGD becomes SGD. The  $O(\sqrt{T})$  regret translates to  $O\left(\frac{1}{\sqrt{T}}\right)$  convergence rate for the average iterate. This connection explains why SGD works despite noisy gradients.



### 🏗️ Scaffolded Derivation (4 steps)

Let's build intuition for the OGD regret bound by deriving it with decreasing support.

**Step 1 (Full support):** Derive the bound for  $\mathcal{K} = \mathbb{R}^d$  (no constraints).

Starting from convexity:  $f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle$

Update rule:  $x_{t+1} = x_t - \eta \nabla f_t(x_t)$ , so  $\nabla f_t(x_t) = \frac{x_t - x_{t+1}}{\eta}$

Therefore:  $\langle \nabla f_t(x_t), x_t - x^* \rangle = \frac{1}{\eta} \langle x_t - x_{t+1}, x_t - x^* \rangle$

Using  $2\langle a, b \rangle = \|a\|^2 + \|b\|^2 - \|a - b\|^2$ :

$$2\langle x_t - x_{t+1}, x_t - x^* \rangle = \|x_t - x^*\|^2 + \|x_t - x_{t+1}\|^2 - \|x_{t+1} - x^*\|^2$$

Since  $\|x_t - x_{t+1}\| = \eta \|\nabla f_t(x_t)\| \leq \eta G$ :

$$f_t(x_t) - f_t(x^*) \leq \frac{\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2}{2\eta} + \frac{\eta G^2}{2}$$

Summing and telescoping:  $R_T \leq \frac{\|x_1 - x^*\|^2}{2\eta} + \frac{\eta G^2 T}{2}$

**Step 2 (Partial support):** Now add constraints  $\mathcal{K} = \{x : \|x\| \leq B\}$ .

**Hint:** The only change is the update becomes  $x_{t+1} = \Pi_{\mathcal{K}}(x_t - \eta \nabla f_t(x_t))$ . How does projection affect the distance  $\|x_{t+1} - x^*\|^2$ ?

**Key insight:** Projection is non-expansive:  $\|\Pi_{\mathcal{K}}(y) - x^*\| \leq \|y - x^*\|$  for any  $x^* \in \mathcal{K}$ .

**Your turn:** Modify the telescoping argument to incorporate projection.

**Step 3 (Minimal support):** Derive the bound for box constraints  $\mathcal{K} = [a_1, b_1] \times \dots \times [a_d, b_d]$ .

**Hint:** Projection onto a box is coordinate-wise clipping. The telescoping argument works the same way.

**Step 4 (Transfer):** Apply the technique to simplex constraints  $\mathcal{K} = \{x : x_i \geq 0, \sum_i x_i = 1\}$ .

**Challenge:** The projection onto simplex is more complex (requires sorting). But the regret bound has the same form! Why doesn't the projection algorithm affect the regret?

## Follow-The-Regularized-Leader (FTRL)

We've now seen two powerful frameworks: MWU for discrete experts using exponential weights, and OGD for continuous spaces using gradients. They seem quite different—one multiplies weights, the other subtracts gradients. But this raises a compelling question: **are they really different algorithms, or two faces of the same principle?**

**The unifying revelation:** FTRL provides a unified framework that encompasses both MWU and OGD. The key insight: both algorithms minimize cumulative loss plus a stabilizing regularizer. The choice of regularizer determines which algorithm you get—quadratic regularization gives OGD-like behavior, entropy regularization gives exponential weights.

**Why unification matters:** FTRL isn't just mathematical elegance. It's a design principle: to create an online learning algorithm for a new domain, choose a regularizer that matches the geometry of your decision space. Simplex? Use entropy. Euclidean space? Use quadratic. This framework has generated dozens of practical algorithms.

FTRL provides a unified framework that is closely related to both MWU and OGD. Some equivalences are exact only after replacing losses by their linearizations.

**Progress on backbone:** We're seeing a profound pattern: seemingly different algorithms (exponential weights, gradient descent) are instances of the same abstract principle—regularized leader-following. This abstraction is the hallmark of mature theory: finding the essential structure beneath surface differences.

### Mental Model: FTRL as Universal Framework

FTRL unifies online learning through the lens of regularization:

**Core Idea:**  $x_t = \arg \min_{x \in \mathcal{K}} \left\{ \eta \sum_{s=1}^{t-1} \langle g_s, x \rangle + R(x) \right\}$

**Regularizer Choice Determines Algorithm:**

Quadratic  $R(x) = \frac{1}{2} \|x\|^2 \xrightarrow{\text{Euclidean geometry}} \text{Dual space} \xrightarrow{\text{Best for Euclidean spaces } (\mathbb{R}^d)}$

Negative Entropy  $R(p) = \sum_i p_i \ln p_i \xrightarrow{\text{Exponential weights}} \text{MWU-style algorithms} \xrightarrow{\text{Best for probability simplex}}$

$\ell_1$  Squared  $R(x) = \frac{1}{2} \|x\|_1^2 \xrightarrow{\text{Sparse updates}} \text{Sparse OGD} \xrightarrow{\text{Best for high-dimensional sparse problems}}$

**Unifying Principle:** Regularizer  $R$  encodes geometry of decision space

- Strong convexity  $\rightarrow$  stability
- Domain-matched  $\rightarrow$  efficiency
- Bregman divergence  $\rightarrow$  mirror descent connection

**Why this matters:**

- Design algorithms by choosing  $R$  for your domain
- Proof technique transfers across all choices
- Same  $O(\sqrt{T})$  regret with appropriate regularizer

### Notation convention

We use  $R(x)$  for the general regularizer on arbitrary convex sets  $\mathcal{K}$ . When specializing to the probability simplex  $\Delta_n$ , we write  $R(p)$  to emphasize that  $p$  is a probability vector. Both refer to the same mathematical object—the regularizer function.

## The FTRL Framework

**Motivation:** Why another algorithm when we already have MW? It's just another algorithm—it's a unifying lens that reveals deep connections. With regularizer  $R$ , we can recover different algorithms and understand why.

**Concrete example first:** Imagine you're repeatedly choosing investments. "Follow the leader" would mean: "invest in whatever portfolio had the lowest loss yesterday." But this is unstable—one bad day could make you switch all your money.

**Regularization** adds inertia: "choose the portfolio that balances low loss and low regularization." The regularizer  $R(x)$  measures "how extreme" a portfolio  $x$  is.

### Gradient vs subgradient notation

We write  $g_t \in \partial f_t(x_t)$  to denote a subgradient from the subdifferential. When  $f_t$  is differentiable at  $x_t$ , the subdifferential is a singleton:  $\partial f_t(x_t) = \{\nabla f_t(x_t)\}$ , so we can write  $g_t = \nabla f_t(x_t)$ .

In practice, most loss functions in OCO are differentiable almost everywhere, so we often use  $\nabla$  notation.

### Definition: Follow-The-Regularized-Leader (FTRL)

**Parameters:** Regularizer  $R : \mathcal{K} \rightarrow \mathbb{R}$  (a convex function that penalizes "complex" or "extreme" decisions), learning rate  $\eta > 0$

**Standard FTRL (Linearized Losses):** For each round  $t = 1, \dots, T$ :

1. **Play:**

$$x_t = \arg \min_{x \in \mathcal{K}} \left\{ \eta \sum_{s=1}^{t-1} \langle g_s, x \rangle + R(x) \right\} \quad (17)$$

where  $g_s \in \partial f_s(x_s)$  are past subgradients

2. **Observe:** Loss function  $f_t$ , compute subgradient  $g_t \in \partial f_t(x_t)$

3. **Incur loss:**  $f_t(x_t)$

**Alternative: FTRL with Nonlinear Losses** FTRL can also be defined using the original nonlinear losses:

$$x_t = \arg \min_{x \in \mathcal{K}} \left\{ \eta \sum_{s=1}^{t-1} f_s(x) + R(x) \right\} \quad (18)$$

However, the linearized version (using gradients) has cleaner analysis and is the standard formulation.

### Intuition:

- "Follow the leader": minimize cumulative past loss
- "Regularized": add  $R(x)$  to stabilize and prevent overfitting
- Regularizer encodes prior knowledge or structural assumptions

### Construct Your Own

**Task:** Design a regularizer for a specific application.

**Scenario:** Online advertising. You allocate budget  $x \in \mathbb{R}^n$  across  $n$  ad channels, where  $x_i \geq 0$  and  $\sum_i x_i = 1$  (simplex). Each round, you observe click-through rates (losses).

**Requirements:**

1. Choose a regularizer  $R(x)$  appropriate for the simplex
2. Explain why your choice makes sense (what behavior does it encourage?)
3. Derive the FTRL update for your regularizer (what's the argmin?)
4. Compare to uniform initialization ( $x_1 = (\frac{1}{n}, \dots, \frac{1}{n})$ )

**Hint:** Think about what properties you want: exploration vs. exploitation, sparsity vs. diversification, fast vs. cautious adaptation.

**Extension:** What if you want to encourage exploration (avoid putting all budget in one channel)? What regularizer achieves this?

## Connection to OGD

### Theorem: FTRL with Quadratic Regularizer = OGD

Assume:

- **Regularizer:**  $R(x) = \frac{1}{2}\|x\|^2$  (quadratic, Euclidean norm)
- **Losses:** Linearized losses  $\langle g_t, x \rangle$  where  $g_t \in \partial f_t(x_t)$
- **Decision space:** Unconstrained  $\mathcal{K} = \mathbb{R}^d$  or Euclidean ball

Then FTRL with learning rate  $\eta$  produces updates similar to online gradient descent:

$$x_t = x_1 - \eta \sum_{s=1}^{t-1} g_s \quad (19)$$

**Interpretation:** This is dual averaging (cumulative gradient descent). OGD uses the recursive form  $x_{t+1} = x_t - \eta g_t$ , which is equivalent when starting from the same point. The general FTRL framework allows other regularizers (e.g., negative entropy for simplex).

**Proof.** For linearized losses  $\langle g_s, x \rangle$ , the FTRL update is:

$$x_t = \arg \min_{x \in \mathcal{K}} \left\{ \eta \sum_{s=1}^{t-1} \langle g_s, x \rangle + \frac{1}{2} \|x - x_1\|^2 \right\} \quad (20)$$

Ignoring constraints for intuition, the first-order condition gives:

$$x_t = x_1 - \eta \sum_{s=1}^{t-1} g_s. \quad (21)$$

This is a cumulative-gradient or dual-averaging update. OGD instead uses the local recursion  $x_{t+1} = \Pi_{\mathcal{X}}(x_t - \eta g_t)$ . Both use Euclidean geometry and achieve the same  $O(\sqrt{T})$  regret under the usual assumptions, but exact equality requires special cases. ■

## Connection to MWU

### Theorem: FTRL with Negative Entropy = MWU

Assume:

- **Decision space:** Probability simplex  $\Delta_n = \{p \in \mathbb{R}^n : p_i \geq 0, \sum_i p_i = 1\}$
- **Regularizer:** Negative entropy  $R(p) = \sum_{i=1}^n p_i \ln p_i$
- **Losses:** Linearized losses  $\langle m_t, p \rangle$  where  $m_t \in [0, 1]^n$  are expert losses

Then FTRL with learning rate  $\eta$  produces the exponential weights update:

$$p_t(i) \propto \exp\left(-\eta \sum_{s=1}^{t-1} m_s(i)\right) \quad (22)$$

This is exactly the Multiplicative Weights Update algorithm from Week 8.

**Interpretation:** MWU is the natural gradient descent on the simplex when using the KL divergence geometry (induced by negative entropy regularization). This explains why MWU is the “right” algorithm for expert advice—it respects the geometry of the probability simplex.

**Proof.** The FTRL update is:

$$p_t = \arg \min_{p \in \Delta_n} \left\{ \eta \sum_{s=1}^{t-1} \langle m_s, p \rangle + \sum_{i=1}^n p_i \ln p_i \right\} \quad (23)$$

Using Lagrange multipliers for the constraint  $\sum_i p_i = 1$ :

$$\eta \sum_{s=1}^{t-1} m_s(i) + \ln p_t(i) + 1 + \lambda = 0 \quad (24)$$

Solving:

$$p_t(i) = \exp\left(-\eta \sum_{s=1}^{t-1} m_s(i) - 1 - \lambda\right) \quad (25)$$

Normalizing to ensure  $\sum_i p_t(i) = 1$ :

$$p_t(i) = \frac{\exp\left(-\eta \sum_{s=1}^{t-1} m_s(i)\right)}{\sum_j \exp\left(-\eta \sum_{s=1}^{t-1} m_s(j)\right)} \quad (26)$$

This can be rewritten as:

$$p_t(i) = w_t \frac{i}{\Phi_t} \quad \text{where} \quad w_t(i) = \prod_{s=1}^{t-1} \exp(-\eta m_s(i)) \quad (27)$$

Using  $\exp(-\eta m_s(i)) \approx 1 - \eta m_s(i)$  for small  $\eta$ , we recover the multiplicative update above. Thus entropy FTRL is exactly exponential weights and approximates the update above.

This is the standard FTRL template: strong convexity of  $R$  ensures stability, and the dual-norm bound controls each linearized loss. The  $O(\sqrt{T})$  rate is universal across all strongly convex regularizers.

## FTRL Regret Analysis

### Definition: Dual Norm

Given a norm  $\|\cdot\|$  on  $\mathbb{R}^d$ , the **dual norm**  $\|\cdot\|_*$  is defined as:

$$\|z\|_* = \sup_{\|x\| \leq 1} \langle z, x \rangle \quad (28)$$

The dual norm satisfies Hölder's inequality: for all  $x, z \in \mathbb{R}^d$ ,

$$\langle z, x \rangle \leq \|z\|_* \cdot \|x\| \quad (29)$$

### Common examples:

- Euclidean norm is self-dual: if  $\|\cdot\| = \|\cdot\|_2$ , then  $\|\cdot\|_* = \|\cdot\|_2$
- $\ell^1$ - $\ell^\infty$  duality: if  $\|\cdot\| = \|\cdot\|_1$ , then  $\|\cdot\|_* = \|\cdot\|_\infty$
- $\ell^\infty$ - $\ell^1$  duality: if  $\|\cdot\| = \|\cdot\|_\infty$ , then  $\|\cdot\|_* = \|\cdot\|_1$

### Theorem: FTRL Regret Bound

Assume:

- **Linearized losses:** FTRL plays  $x_t$  and observes gradient  $g_t \in \partial f_t(x_t)$ , incurring linearized loss  $\langle g_t, x_t \rangle$
- **Strongly convex regularizer:**  $R : \mathcal{K} \rightarrow \mathbb{R}$  is  $\alpha$ -strongly convex with respect to norm  $\|\cdot\|$
- **Bounded dual norm:**  $\|g_t\|_* \leq G$  for all  $t \in [T]$  (where  $\|\cdot\|_*$  is the dual norm)
- **Learning rate:**  $\eta > 0$

Then for any comparator  $x^* \in \mathcal{K}$ :

$$R_T = \sum_{t=1}^T \langle g_t, x_t \rangle - \sum_{t=1}^T \langle g_t, x^* \rangle \leq \frac{R(x^*) - R(x_1)}{\eta} + \frac{\eta G^2 T}{2\alpha} \quad (30)$$

**Optimal tuning:** If  $R(x^*) - R(x_1) \leq B$ , setting  $\eta = \sqrt{\frac{2\alpha B}{G^2 T}}$  gives:

$$R_T \leq \sqrt{\frac{2BG^2 T}{\alpha}} \quad (31)$$

**Optimal Learning Rate:** Setting  $\eta = \sqrt{\frac{2\alpha(R(x^*) - R(x_1))}{G^2 T}}$  (requires knowing  $T$  in advance) gives:

$$R_T = O(\sqrt{T}) \quad (32)$$

**Example: Choosing Regularizers**

Different regularizers lead to different algorithms:

| Regularizer            | Domain         | Algorithm          |
|------------------------|----------------|--------------------|
| $\frac{1}{2}\ x\ ^2$   | $\mathbb{R}^d$ | OGD                |
| $\sum_i p_i \ln p_i$   | Simplex        | MWU                |
| $\frac{1}{2}\ x\ _1^2$ | $\mathbb{R}^d$ | Sparse OGD         |
| $\sum_i \sqrt{p_i}$    | Simplex        | Polynomial Weights |

**Comparison: OGD vs. FTRL vs. MWU**

| Property    | OGD                | FTRL                      | MWU                                    |
|-------------|--------------------|---------------------------|----------------------------------------|
| Domain      | Convex set         | Convex set                | Simplex                                |
| Update      | Gradient step      | Minimize regularized loss | Multiplicative                         |
| Regularizer | Euclidean geometry | Explicit $R(x)$           | Negative entropy / exponential weights |
| Regret      | $O(\sqrt{T})$      | $O(\sqrt{T})$             | $O(\sqrt{T})$                          |
| Projection  | Required           | Via optimization          | Implicit                               |

**Connection****Unified View:**

- OGD: local updates via gradients
- FTRL: global optimization with regularization
- MWU: entropy FTRL gives exponential weights on simplex; the linear update is a close first-order variant

All achieve  $O(\sqrt{T})$  regret, but differ in:

- Computational efficiency
- Ease of implementation
- Suitability for different domains

**Applications of FTRL**

**Example: Zero-Sum Games (Sketch)**

**Problem:** Two-player zero-sum game with payoff matrix  $A \in \mathbb{R}^{n \times m}$  to Player 1.

- Player 1 chooses row  $i$  (or mixed strategy  $p \in \Delta_n$ ) and wants to maximize payoff
- Player 2 chooses column  $j$  (or mixed strategy  $q \in \Delta_m$ ) and wants to minimize payoff
- Player 1's payoff:  $A(i, j)$  or  $p^\top Aq$  for mixed strategies

**Nash Equilibrium:**  $(p^*, q^*)$  has value  $v$  such that:

$$\min_j (p^*)^\top A e_j = v = \max_i e_i^\top A q^* \quad (33)$$

**FTRL Approach:**

- Both players run entropy-based no-regret algorithms
- Player 1 can minimize loss  $f_t(p) = -p^\top Aq_t$
- Player 2 can minimize loss  $g_t(q) = p_t^\top Aq$
- After  $T$  rounds, average strategies  $(\bar{p}, \bar{q})$  approximate Nash equilibrium

**Convergence:** The saddle-point gap is  $O\left(\frac{1}{\sqrt{T}}\right)$  under bounded payoffs.

**Example: Linear Programming (Sketch)**

**Problem:** Covering LP with non-negative coefficients

$$\min \sum_j c_j x_j \quad \text{s.t.} \quad \sum_j a_{ij} x_j \geq 1 \quad \forall i, \quad x_j \geq 0 \quad (34)$$

**FTRL Approach:**

- Each constraint  $i$  is an “expert”
- Maintain weights  $w_t(i)$  on constraints
- Update weights based on constraint violations
- Converges to approximate feasible solution

**Guarantee:** With the right oracle and bounded-width assumptions, MWU-style methods can obtain  $(1 + \varepsilon)$ -approximations in  $O\left(\frac{\log n}{\varepsilon^2}\right)$  iterations.

**→ Week 12 Preview: Optimization Landscape of Neural Networks**

Week 12 explores why neural networks have “nice” optimization landscapes despite non-convexity. One key insight: overparameterized networks have many global minima, and SGD (= OGD on stochastic batches) finds them efficiently. The regret analysis we developed—showing OGD converges even with adversarial losses—provides theoretical foundation for understanding SGD's success in non-convex settings.



### Author's Take

FTRL is a general framework for online learning. By choosing different regularizers and, when needed, linearizing losses, we recover or closely relate to different algorithms:

- Quadratic regularizer  $\rightarrow$  Euclidean dual averaging / close relative of OGD (good for Euclidean spaces)
- Entropy regularizer  $\rightarrow$  exponential weights / MWU-style algorithms (good for probability simplices)
- Other regularizers  $\rightarrow$  specialized algorithms for specific domains

The key insight: regularization provides stability and prevents overfitting to past losses.

### ✓ Checkpoint

**Checkpoint 8:** Why does FTRL need regularization? What happens without it (i.e.,  $R(x) = 0$ )?

**Answer:** Without regularization, FTRL becomes “Follow-The-Leader”:  $x_t = \arg \min_{x \in \mathcal{K}} \sum_{s=1}^{t-1} \langle g_s, x \rangle$ . This can be unstable—small changes in losses cause large swings in  $x_t$ . FTL can have  $\Theta(T)$  regret! Regularization adds inertia, preventing wild oscillations and ensuring  $O(\sqrt{T})$  regret.

### 🧩 Synthesis Challenge

**Challenge:** Prove that Follow-The-Leader (FTL) without regularization can have linear regret.

**Setup:**  $\mathcal{K} = [-1, 1]$ ,  $T = 2n$  rounds, losses alternate:  $g_t = +1$  for odd  $t$ ,  $g_t = -1$  for even  $t$ .

**Your tasks:**

1. Compute  $x_t = \arg \min_{x \in [-1, 1]} \sum_{s=1}^{t-1} g_s \cdot x$  for each round
2. Calculate total loss:  $\sum_{t=1}^{2n} g_t x_t$
3. Calculate best fixed point loss:  $\min_{x \in [-1, 1]} \sum_{t=1}^{2n} g_t x$
4. Show regret is  $\Theta(T)$ , not  $O(\sqrt{T})$

**Why this matters:** This proves that regularization isn't just a mathematical trick—it's essential for sublinear regret. The  $O(\sqrt{T})$  bound requires stability, which regularization provides.

### Key Takeaway

FTRL unifies many online learning algorithms through the lens of regularization. OGD, FTRL, and MWU share the same stability-via-regularization principle, though exact equivalences depend on whether losses are linearized and on the chosen update form. The framework provides a principled way to design online algorithms: choose a regularizer appropriate for your domain, then minimize cumulative linearized loss plus regularization.

## ✓ Checkpoint

**Checkpoint 9:** The FTRL regret bound is  $\frac{R(x^*) - R(x_1)}{\eta} + \frac{\eta G^2 T}{2\alpha}$ . How does strong convexity parameter  $\alpha$  affect regret?

**Answer:** Larger  $\alpha$  (more strongly convex  $R$ ) reduces the second term  $\frac{\eta G^2 T}{2\alpha}$ —stronger regularization better controls gradient noise. But it doesn't affect the first term much. The optimal  $\eta$  balances both terms, giving  $O(\sqrt{T})$  regret regardless of  $\alpha$  (though the constant depends on  $\alpha$ ).

 Theorem Landscape

This section presented three connection results showing FTRL unifies online learning:

- **Theorem 5.1 (FTRL with Quadratic Regularizer):** Closely related to OGD via dual averaging
- **Theorem 5.2 (FTRL with Negative Entropy):** Gives exponential weights; MWU is first-order approximation
- **Theorem 5.3 (General FTRL Regret):** Template bound for any strongly convex regularizer

**Architecture:** Different regularizers  $\rightarrow$  different algorithms, same proof technique

**Key insight:** The geometry of the decision space determines the right regularizer:

- Euclidean space  $\rightarrow$  quadratic regularizer  $\rightarrow$  OGD
- Probability simplex  $\rightarrow$  negative entropy  $\rightarrow$  MWU
- Sparse vectors  $\rightarrow \ell_1$ -based regularizer  $\rightarrow$  sparse updates

**Practical impact:** To design an online algorithm for a new domain, choose  $R$  that matches the geometry, then apply FTRL. The  $O(\sqrt{T})$  regret follows automatically if  $R$  is strongly convex.

## Lower Bounds and Optimality

This lower bound shows that MWU, OGD, and FTRL are optimal up to constant factors. We cannot hope for better than  $O(\sqrt{T})$  regret in general.

We've seen algorithms achieving  $O(\sqrt{T})$  regret. But this raises a critical question: **is this optimal, or can we do better?** Could there be a clever algorithm that achieves  $O(\log T)$  or even  $O(1)$  regret in the adversarial setting?

**The fundamental limits:** The  $O(\sqrt{T})$  rate isn't arbitrary—it's fundamental. Just as VC theory revealed  $\Theta(\frac{d}{\epsilon})$  sample complexity is unavoidable, online learning theory shows  $\Omega(\sqrt{T})$  regret is unavoidable. This lower bound proves our algorithms are optimal up to constants and logarithmic factors.

**Why this matters:** Upper bounds show what's achievable; lower bounds show what's impossible. Together, they bracket the truth. Without lower bounds, we'd never know if we should keep searching for better algorithms or if we've already found the best possible.

**Progress on backbone:** This completes the online learning story: we've identified the right question (minimize regret), found optimal algorithms (MWU, OGD, FTRL), and proven they cannot be improved asymptotically. This mirrors the completeness of VC theory for PAC learning—both theories fully characterize their settings.

### ← From Week 2: Lower Bounds and Information Theory

Week 2's lower bounds for PAC learning used counting arguments: you need enough samples to distinguish  $|\mathcal{H}|$  hypotheses. Online learning lower bounds use a different technique—adversarial construction—but the spirit is similar: information limits what's achievable. You can't predict the future perfectly without observing enough rounds.

## Information-Theoretic Lower Bound

**Motivation:** Upper bounds show what's achievable; lower bounds show what's impossible. Together, they bracket the truth. Our  $O(\sqrt{T})$  upper bounds would be unsatisfying without knowing if  $O(\log T)$  or  $O(1)$  regret is possible.

### Theorem: Lower Bound for Online Learning

In adversarial expert advice with bounded losses, and more generally in bounded-domain Lipschitz online convex optimization, every algorithm has worst-case regret

$$R_T = \Omega(\sqrt{T}) \quad (35)$$

for some loss sequence.

**Proof. Why the naive adversary is not enough:** Consider the expert advice problem with  $n = 2$  experts.

### Adversarial Construction:

- In each round  $t$ , adversary observes algorithm's distribution  $p_t = (p_t(1), p_t(2))$
- Adversary sets losses to make algorithm perform poorly:

- If  $p_t(1) \geq \frac{1}{2}$ : set  $m_t(1) = 1, m_t(2) = 0$
- If  $p_t(1) < \frac{1}{2}$ : set  $m_t(1) = 0, m_t(2) = 1$

**Algorithm's Expected Loss:**

$$\mathbb{E}[L_T] = \sum_{t=1}^T \max(p_t(1), p_t(2)) \geq \frac{T}{2}$$

**Key insight**

The  $\sqrt{T}$  lower bound is information-theoretic: any learner needs time to learn which expert/action is best, and during this learning phase, regret accumulates unavoidably.

**Best Expert's Loss:** Best expert has loss at most  $\frac{T}{2}$  (one expert is right at least half the time).

**Regret:**

$$R_T \geq \frac{T}{2} - \frac{T}{2} = 0 \quad (37)$$

This seems to give no lower bound! The issue is we need a more careful construction that prevents the learner from quickly identifying the best action.

**Refined Argument (Information-Theoretic):**

We sketch the information-theoretic argument. The full proof uses Yao's minimax lemma to convert worst-case regret to average-case regret against a random adversary.

**Step 1: Random adversary construction**

Consider  $n = 2$  experts. The adversary secretly chooses a random “good” expert  $i^* \in \{1, 2\}$  uniformly at random, and sets losses:

- Expert  $i^*$  has expected loss  $\frac{1}{2} - \varepsilon$  per round
- The other expert has expected loss  $\frac{1}{2} + \varepsilon$  per round
- Losses are stochastic with bounded variance  $\sigma^2$

Against this random adversary, any learner must distinguish which expert is better.

**Step 2: Information-theoretic limit**

From statistical estimation theory, identifying the better expert requires observing  $\Omega\left(\frac{\sigma^2}{\varepsilon^2}\right)$  samples. This is the fundamental limit of hypothesis testing with noisy observations.

**Step 3: Balance exploration and regret**

During the identification phase, the learner accumulates regret. Setting  $\varepsilon = \frac{1}{\sqrt{T}}$  balances:

- Need  $\Omega(T)$  rounds to identify the better expert with high probability
- Regret per round during exploration:  $\Omega(\varepsilon)$
- Total regret:  $\Omega(\varepsilon T) = \Omega(\sqrt{T})$

**Step 4: Extension to continuous domains**

For OCO with Lipschitz losses on a bounded domain, a similar construction uses a random “good” direction. The learner must identify the direction of the negative gradient, which again requires  $\Omega(\sqrt{T})$  regret.

**Formal proof:** See Cesa-Bianchi & Lugosi (2006), “Prediction, Learning, and Games”, Theorem 3.7 for the complete argument using Yao's minimax lemma. ■

**Corollary: Optimality of MWU and OGD**

MWU and OGD achieve regret  $O(\sqrt{T \ln n})$  and  $O(\sqrt{T})$  respectively, matching the lower bound up to logarithmic factors. These algorithms are **minimax optimal**.

**✓ Checkpoint**

**Checkpoint 10:** If someone claims they have an algorithm with  $O(\log T)$  regret in the adversarial expert advice setting with no additional assumptions, what can you conclude?

**Answer:** They're either wrong, or they're using stronger assumptions than stated. The  $\Omega(\sqrt{T})$  lower bound is unconditional in the adversarial setting.  $O(\log T)$  is only possible with additional structure: stochastic losses with gaps, strongly convex losses, or exp-concavity.

**Impossibility Results****Remark****What's Impossible:**

- $o(\sqrt{T})$  regret in adversarial setting (without additional assumptions)
- Competing with best sequence of experts (not just best single expert)
- Achieving  $O(\log T)$  regret without additional stochastic gap, curvature, or exp-concavity assumptions

**What's Possible with Stronger Assumptions:**

- Stochastic losses with gaps or exp-concavity:  $O(\log T)$  regret possible
- Strongly convex losses:  $O(\log T)$  regret possible
- Bandit feedback:  $O(\sqrt{nT})$  regret (worse by  $\sqrt{n}$  factor)

 **Synthesis Challenge**

**Challenge:** Design an online learning algorithm for a novel setting.

**Scenario:** You're building a recommendation system that must adapt to user preferences. Each round:

1. You recommend one of  $n$  items
2. User either clicks (reward = 1) or doesn't (reward = 0)
3. You only observe feedback for the item you recommended (bandit feedback)
4. User preferences drift slowly over time (non-stationary)

**Your tasks:**

1. Is this adversarial or stochastic? Justify.
2. Can you use MWU directly? If not, what modification is needed?
3. The “bandit” constraint (only observing chosen item) is problematic. How would you estimate losses for unchosen items?
4. Design an algorithm and argue it achieves sublinear regret.

**Hints:**

- For bandit feedback, use importance-weighted estimators:  $\tilde{m}_{t(i)} = \frac{m_{t(i)}}{p_{t(i)}}$  if  $i$  chosen, 0 otherwise
- For non-stationarity, use a sliding window or increase exploration
- Your regret bound will have  $\sqrt{nT}$  instead of  $\sqrt{T}$  due to bandit constraint

**Why this matters:** This is the EXP3 algorithm—used in real recommendation systems, online advertising, and A/B testing!

## Connections Across the Course

We’ve completed our exploration of online learning—from binary expert advice to continuous convex optimization, from deterministic algorithms to randomized ones, from specific algorithms to the unifying FTRL framework. Now let’s step back and see how online learning fits into the broader course narrative.

**The grand synthesis:** We’ve journeyed from PAC learning (Weeks 1-3) through boosting (Weeks 6-7), and now online learning. A beautiful pattern emerges: the multiplicative weight update principle—exponentially reward good performers, exponentially penalize bad ones—unifies AdaBoost, MWU, and even modern deep learning optimizers. This isn’t coincidence; it’s a fundamental algorithmic pattern for learning under uncertainty.

**Progress on backbone:** Online learning completes a conceptual arc: PAC learning asks “how many samples suffice?”, boosting asks “can weak learners be combined?”, and online learning asks “can we adapt without distributional assumptions?” Each framework tackles uncertainty differently, but all rely on fundamental principles: regularization prevents overfitting, convexity enables optimization, and exponential updates track changing performance.

### Connection

#### Week 7 AdaBoost → Week 9 MWU

AdaBoost maintains distribution  $D_t$  over training examples:

$$D_{t+1}(i) \propto D_t(i) \exp(-\alpha_t y_i h_t(x_i)) \quad (38)$$

This closely mirrors exponential weights with:

- Training examples = experts
- Weak hypotheses = loss vectors
- Distribution  $D_t$  = weight distribution  $w_t$

**Key Insight:** Boosting can be viewed as online learning over training examples, using an exponential update with example-dependent losses.

### Connection

#### Week 1 PAC Learning → Week 9 Online Learning

| PAC Learning         | Online Learning       |
|----------------------|-----------------------|
| i.i.d. data          | Adversarial sequences |
| Generalization error | Regret                |
| Sample complexity    | Time complexity       |
| Uniform convergence  | No-regret property    |
| VC dimension         | Number of experts     |

Both frameworks study learning, but with different assumptions and guarantees.

### Connection

#### Week 9 Online Learning → Week 11-12 Deep Learning

Modern deep learning uses online optimization:

- **SGD:** Online gradient descent on mini-batches
- **Adam:** Adaptive learning rates (like adaptive MWU)
- **Training dynamics:** Sequential updates on data stream

Online learning theory provides foundations for understanding SGD convergence and generalization.

#### → Week 11 Preview: Neural Tangent Kernels

Week 11 introduces the Neural Tangent Kernel (NTK) theory, which analyzes neural network training in the infinite-width limit. The OGD analysis we developed—tracking  $\|x_t - x^*\|^2$  through gradient updates—directly extends to NTK regime. The  $O(\sqrt{T})$  convergence rate carries over, explaining why overparameterized networks trained with SGD generalize well despite zero training loss.

#### → Week 12 Preview: Generalization in Deep Learning

Week 12 asks: Why do neural networks generalize despite having more parameters than training examples? Part of the answer lies in implicit regularization—SGD doesn't just minimize loss, it implicitly optimizes a regularized objective. This connects to FTRL: the stochastic noise in SGD acts like entropy regularization, preferring “flat” minima that generalize better.

### Synthesis Challenge

**Challenge:** Connect three course concepts through a unified narrative.

**Task:** Write a one-page explanation (for a peer) connecting:

1. VC dimension and sample complexity (Week 2)
2. Boosting and ensemble learning (Week 9)
3. Online learning and regret bounds (Week 10)

#### Requirements:

- Explain how each builds on the previous
- Identify the common thread (hint: it's about combining weak learners/predictions)
- Show how the mathematical techniques evolve (uniform convergence → potential functions → regret analysis)
- Conclude with why this progression matters for modern ML

**Bonus:** Predict what Week 11-12 will add to this story (you don't know yet, but can guess from the trajectory).



## Practice Problems

### Warm-up Problems (★)

#### Student Challenge

**Problem 1:** Consider the Weighted Majority Algorithm with  $n = 3$  experts and  $\eta = 0.5$ . Starting with weights  $(1, 1, 1)$ , compute the weights after 3 rounds where:

- Round 1: Experts 1 and 2 are wrong
- Round 2: Expert 3 is wrong
- Round 3: Expert 1 is wrong

What is the weighted majority prediction in round 4?

#### Student Challenge

**Problem 2:** Prove that for  $x \in [0, 1]$ :  $1 - x \leq e^{-x} \leq 1 - x + \frac{x^2}{2}$ .

Use these inequalities to derive a regret bound for MWU.

#### Student Challenge

**Problem 3:** Show that the projection onto the Euclidean ball  $\mathcal{K} = \{x : \|x\| \leq R\}$  is:

$$\Pi_{\mathcal{K}}(y) = \begin{cases} y & \text{if } \|y\| \leq R \\ \frac{Ry}{\|y\|} & \text{if } \|y\| > R \end{cases} \quad (39)$$

### Intermediate Problems (★★)

#### Student Challenge

**Problem 4:** Signed losses are often handled by exponential weights. Suppose  $m_t(i) \in [-1, 1]$  and use  $w_{t+1}(i) = w_{t(i)} \exp(-\eta m_{t(i)})$ . Prove a regret bound of order  $\eta T + \frac{\ln n}{\eta}$ .

#### Student Challenge

**Problem 5:** Prove that the negative entropy function  $R(p) = \sum_i p_i \ln p_i$  is 1-strongly convex on the simplex with respect to the  $\ell_1$  norm.

**Hint:** Show that  $\nabla^2 R(p)$  has eigenvalues bounded below by 1.

**Student Challenge**

**Problem 6:** Consider online linear regression with squared loss. At time  $t$ , you observe  $a_t \in \mathbb{R}^d$  with  $\|a_t\| \leq 1$ , predict  $\langle x_t, a_t \rangle$ , and observe  $b_t \in [-1, 1]$ .

Apply OGD with  $\mathcal{K} = \{x : \|x\| \leq B\}$  and derive the regret bound. What is the optimal step size?

**Advanced Problems (★★★)****Student Challenge****Problem 7: Doubling Trick for Unknown Horizon**

Suppose  $T$  is unknown. Divide time into epochs of lengths  $1, 2, 4, 8, \dots$  and run MWU with  $\eta_k = \sqrt{\ln \frac{n}{2^k}}$  in epoch  $k$ .

Show that the total regret is at most  $O(\sqrt{T \ln n})$ , matching the bound when  $T$  is known.

**Hint:** Sum regret over epochs and use  $\sum_{k=0}^{\log T} \sqrt{2^k} = O(\sqrt{T})$ .

**Student Challenge****Problem 8: Adaptive Regret**

Define the **adaptive regret** over interval  $[s, t]$  as:

$$R_{s,t} = \sum_{\tau=s}^t \ell_{\tau}(\text{learner}) - \min_{i \in [n]} \sum_{\tau=s}^t \ell_{\tau}(\text{expert } i) \quad (40)$$

Standard regret is  $R_{1,T}$ . Adaptive regret requires  $R_{s,t} = o(t - s)$  for all intervals.

Show that standard MWU does **not** achieve adaptive regret. What modification is needed?

**Student Challenge****Problem 9: Bandit Feedback**

In the bandit setting, you only observe the loss of the chosen expert, not all experts' losses.

Consider the EXP3 algorithm: use MWU with estimated losses

$$\tilde{m}_t(i) = \begin{cases} m_t \frac{i}{p_t}(i) & \text{if expert } i \text{ chosen} \\ 0 & \text{otherwise} \end{cases} \quad (41)$$

Show that  $\mathbb{E}[\tilde{m}_t(i)] = m_t(i)$  and derive a regret bound of  $O(\sqrt{nT \ln n})$ .

Why is the dependence on  $n$  worse than full information?

## Conceptual Problems (🤔)

### Student Challenge

#### Problem 10: Stochastic vs. Adversarial

Suppose losses are i.i.d. from some distribution (stochastic setting). Can we achieve better than  $O(\sqrt{T})$  regret?

Research the UCB (Upper Confidence Bound) algorithm and explain how it achieves  $O(\log T)$  regret in the stochastic bandit setting.

### Student Challenge

#### Problem 11: Online Learning in Practice

Consider an online advertising system that must choose which ad to display to each user.

- (a) Formulate this as an online learning problem. What are the experts? What is the loss?
- (b) Would you use MWU, OGD, or FTRL? Justify your choice.
- (c) What practical considerations (computational cost, memory, interpretability) affect your choice?

### Student Challenge

#### Problem 12: Connection to Game Theory

In a repeated zero-sum game, if both players use MWU, their average strategies converge to Nash equilibrium.

- (a) Explain intuitively why this happens.
- (b) What is the convergence rate?
- (c) How does this relate to the minimax theorem?

## Summary and Looking Ahead

We began this week with a deceptively simple question: can we make good predictions when the world is adversarial? No probability distributions, no i.i.d. assumptions—just an opponent trying to make us fail. The journey from that question to its answer has been remarkable.

**What we’ve accomplished:** Starting from the Halving Algorithm’s binary elimination, we’ve built up a complete theory of online learning. We’ve seen how randomization breaks impossibility barriers, how exponential weights achieve optimal regret, how the same principles apply in both discrete and continuous spaces, and how FTRL unifies everything under one elegant framework.

**The deep lesson:** This week revealed the power of **multiplicative weight updates** for adversarial learning. From Halving’s binary elimination to Weighted Majority’s gradual decay, from randomized sampling to continuous gradient descent, from discrete experts to regularized optimization—all share the exponential reweighting principle. This unifying idea connects classical online learning, modern deep learning, game theory, and optimization.

**Progress on backbone:** Online learning completes a conceptual trilogy with PAC learning and boosting. PAC learning asks “how much data suffices?”, boosting asks “can weak learners be strengthened?”, and online learning asks “can we adapt without assumptions?” The answer to all three is yes, and the techniques that make it possible—uniform convergence, exponential loss, multiplicative weights—form the core toolkit of modern learning theory.

## Key Concepts Covered

### Algorithms:

- Halving Algorithm:  $O(\log n)$  mistakes with perfect expert
- Weighted Majority:  $O(M^* + \sqrt{M^* \log n})$  mistakes
- MWU:  $O(\sqrt{T \log n})$  regret for real-valued losses
- OGD:  $O(\sqrt{T})$  regret for convex optimization
- FTRL: Unified framework encompassing MWU and OGD

### Techniques:

- Potential function analysis
- Multiplicative weight updates
- Projection onto convex sets
- Regularization for stability

### Results:

- $O(\sqrt{T})$  regret is optimal (matches lower bound)
- Randomization improves constants
- No distributional assumptions needed

## ✓ Checkpoint

**Checkpoint 11 (Synthesis):** Why is  $O(\sqrt{T})$  regret “sublinear” and what does this mean intuitively?

**Answer:** Sublinear means  $\frac{R_T}{T} \rightarrow 0$  as  $T \rightarrow \infty$ . Intuitively: the per-round average regret vanishes. After enough rounds, we’re performing as well as the best expert **as if** we had known their identity from the start. We “learn to match” the best in hindsight.

## ✓ Checkpoint

**Checkpoint 12 (Meta-level):** We studied five algorithms this week. Can you organize them by: (1) decision space, (2) update rule, (3) regret bound?

**Answer:**

- Halving: discrete experts, eliminate wrong,  $O(\log n)$  mistakes (perfect expert needed)
- WM: discrete, multiply by  $(1 - \eta)$ ,  $O(\sqrt{M^* \log n})$  mistakes
- RWM: discrete, same update but sample,  $O(\sqrt{M^* \log n})$  expected mistakes
- MWU: discrete/simplex, exponential weights,  $O(\sqrt{T \log n})$  regret
- OGD: continuous convex, gradient + projection,  $O(\sqrt{T})$  regret
- FTRL: any convex, regularized leader,  $O(\sqrt{T})$  regret

## Looking Ahead: Week 11-12

Next, we’ll study **deep learning theory**:

- Neural network expressivity and approximation
- Training dynamics and optimization landscape
- Generalization in overparameterized networks
- Neural Tangent Kernel (NTK) theory

**Connection:** SGD for training neural networks is online gradient descent! The theory we developed this week provides foundations for understanding deep learning optimization.

### → Week 11 Preview: Why Deep Learning Works

Week 11-12 tackle the mystery: Why do overparameterized neural networks (millions of parameters, thousands of examples) generalize? The answer involves:

1. **Optimization:** SGD (= OGD on mini-batches) finds good minima via implicit regularization
2. **Implicit bias:** The  $O(\sqrt{T})$  convergence analysis extends to show SGD prefers “simple” solutions
3. **NTK regime:** In infinite width, networks behave like kernel methods—and we can analyze them!

Everything we learned about OGD, regularization, and convergence rates directly applies.

### Key Takeaway

Online learning provides a robust framework for sequential decision-making without distributional assumptions. The Multiplicative Weights Update method and its variants (OGD, FTRL) achieve optimal  $O(\sqrt{T})$  regret through simple multiplicative updates. These algorithms have applications ranging from game theory to optimization to modern machine learning, and provide theoretical foundations for understanding SGD and adaptive optimizers in deep learning.

**Final backbone:** The exponential reweighting principle— $w \leftarrow we^{-\eta\ell}$  or equivalently  $w \leftarrow w(1 - \eta\ell)$ —is perhaps the most important algorithmic idea in online learning. It appears in expert advice (MWU), boosting (AdaBoost), game theory (fictitious play), convex optimization (OGD/FTRL), and deep learning (SGD/Adam). Recognizing this pattern across domains is key to understanding modern machine learning.

### Confidence Check

**Final Self-Assessment:** Before moving to Week 11, rate your mastery (1-5):

- ☐ I can explain the online learning framework and how it differs from PAC learning
- ☐ I understand why regret  $R_T = o(T)$  is the right goal
- ☐ I can derive the Weighted Majority bound using potential functions
- ☐ I understand why randomization improves from factor 2 to factor 1
- ☐ I can prove the MWU regret bound for real-valued losses
- ☐ I can analyze OGD using telescoping distance arguments
- ☐ I understand FTRL and how regularizer choice determines the algorithm
- ☐ I see the connection between AdaBoost, MWU, OGD, and FTRL
- ☐ I understand why  $O(\sqrt{T})$  regret is optimal
- ☐ I can apply these ideas to new problems (portfolio optimization, game theory, etc.)

**If most are 4-5:** You're ready for deep learning theory! **If most are 3:** Review the proof templates and work through practice problems **If most are 1-2:** Revisit the mental models and backward/forward connections